Fine-Tuning Guide — When, How & Best Practices

> 🎯 **Goal**: Know when fine-tuning is the right approach, how to prepare data, execute training, evaluate results, and avoid common pitfalls.

> ⏱️ **Time**: 2h theory + 2–3h practice

> 📦 **Deliverable**: A mental model for the full fine-tuning lifecycle + runnable data preparation pipeline

---

Table of Contents

1. When to Fine-Tune (and When NOT To)
   - Decision Flowchart
   - Decision Table
   - Common Wrong Reasons
   - Common Right Reasons
2. Data Preparation (The Most Important Step)
   - Data Format
   - Data Quality Checklist
   - Data Preparation Pipeline
   - Common Data Mistakes
3. Fine-Tuning Methods (Practical)
   - OpenAI Fine-Tuning (API-based)
   - LoRA with Hugging Face
   - QLoRA (Budget-Friendly)
   - Method Comparison
4. Hyperparameter Guide
5. Evaluation & Testing
   - Before vs After Comparison
   - Automated Evaluation
   - A/B Testing in Production
   - Red-Teaming
6. Best Practices & Pitfalls
   - Best Practices

- Common Pitfalls
- Cost Estimation

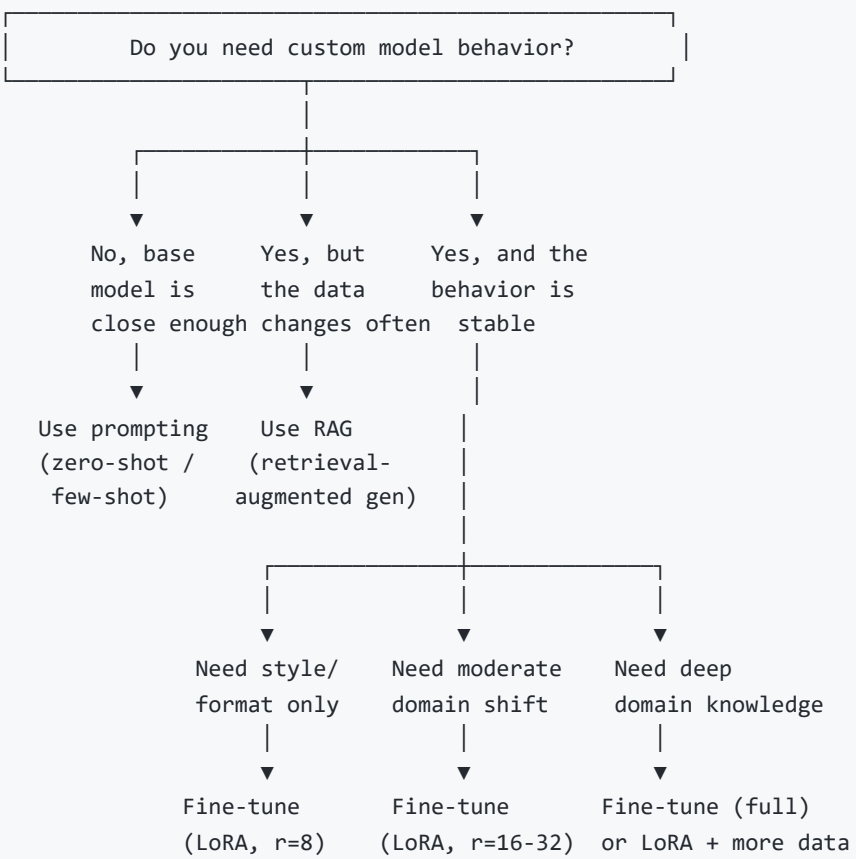
7. Complete Workflow Checklist

When to Fine-Tune (and When NOT To)

Fine-tuning is powerful but **overused**. Most problems that seem like they need fine-tuning actually need better prompts or retrieval. Think of it like renovating a house: don't tear down walls when rearranging furniture would solve the problem.

Decision Flowchart

Before you write a single line of training code, walk through this flowchart:



Decision Table

	Prompting	RAG	LoRA	Full Fine-Tune
Data needed	0 examples	100s of docs	1K–100K	100K+
Cost	~$0	~$10–100/mo	~$10–200	~$100–5000

```
| Time to deploy  | Minutes      | Hours–days   | Hours         | Days–weeks     |
| Quality         | Good (80%)   | Great (90%)  | Great (90%)   | Best (95%+)    |
| When to use     | General tasks| Knowledge-   | Style, format | Deep domain    |
|                 | prototyping  | heavy tasks  | consistency   | specialization |
| Maintenance     | Update prompt| Update docs  | Retrain ~qtr  | Retrain ~qtr   |
└─────────────────┴──────────────┴──────────────┴───────────────┴────────────────┘
```

Common WRONG Reasons to Fine-Tune

Before you spend time and money on fine-tuning, check if your reason is actually on the "wrong" list:

- **"The model doesn't know X"** → Probably needs RAG, not fine-tuning. Fine-tuning teaches *behavior*, not *facts*. For factual knowledge, give the model documents to reference at inference time.
- **"I want better accuracy"** → Try better prompts first. 80% of the time, a well-crafted system prompt with few-shot examples solves the problem. Spend 2 hours on prompt engineering before spending 2 days on fine-tuning.
- **"I want faster responses"** → Fine-tuning doesn't change inference speed. The model has the same number of parameters before and after. If you need speed, use a smaller model or optimize your serving infrastructure.
- **"Everyone else is doing it"** → Fine-tuning is a tool, not a badge of sophistication. The simplest solution that works is always the best one.

Common RIGHT Reasons to Fine-Tune

These are the cases where fine-tuning genuinely shines:

- **Consistent output format/style** — When prompts can't reliably produce the exact structure you need (e.g., always output valid YAML with specific fields, never deviate). Fine-tuning bakes the format into the model's weights.
- **Domain-specific language or jargon** — Medical, legal, financial, or internal company terminology that the base model handles awkwardly. After fine-tuning, the model uses the right terms naturally.
- **Reducing token usage** — Every API call with few-shot examples costs tokens. If you're sending 500 tokens of examples per request at 10K requests/day, fine-tuning pays for itself fast. Encode the behavior instead of demonstrating it every call.
- **Distillation** — Teaching a small, fast model (8B) to behave like a large, expensive model (70B+) for a specific task. You run the big model on your training data, then fine-tune the small model on those outputs. Production costs drop 10×.

Data Preparation (The Most Important Step)

Analogy: Fine-tuning is like teaching someone a new skill by showing them examples. If your examples are sloppy, inconsistent, or wrong — the student learns sloppiness. Data quality is the single biggest determinant of fine-tuning success.

```
The hierarchy of what matters:

    Data Quality          ████████████████████  (50% of outcome)
    Data Diversity        ██████████████        (25% of outcome)
    Hyperparameters       ████████              (15% of outcome)
    Base Model Choice     █████                 (10% of outcome)

500 excellent examples > 50,000 mediocre ones. Every time.
```

Data Format

The standard format for chat-based fine-tuning is **JSONL** (JSON Lines) — one JSON object per line, each representing a complete conversation:

```
{"messages": [{"role": "system", "content": "You are a medical coding assistant. Cla
{"messages": [{"role": "system", "content": "You are a medical coding assistant. Cla
```

```
Key rules for the JSONL format:

┌──────────────────────────────────────────────────────────────┐
│ 1. One JSON object per line — no pretty printing, no newlines │
│ 2. Every example MUST have a "messages" array                │
│ 3. Messages must include at least "user" and "assistant" roles │
│ 4. "system" role is optional but HIGHLY recommended          │
│ 5. The assistant message is what the model learns to produce │
│ 6. Keep the SAME system prompt across ALL examples           │
└──────────────────────────────────────────────────────────────┘
```

Data Quality Checklist

Before you start training, run every single example through this mental filter:

- ☐ **Is this a PERFECT response?** Every assistant message should be exactly what you'd want the model to say. If you wouldn't be proud to show it to a customer, rewrite it.
- ☐ **Is the format consistent?** If example 1 uses bullet points and example 2 uses numbered lists for the same type of response — pick one and fix it.
- ☐ **Does the dataset cover the full input range?** Not just easy, obvious cases. Include short inputs, long inputs, ambiguous inputs, multi-part questions, and edge cases.

- ☐ **Are edge cases represented?** What should the model do when the input is incomplete? When it contains errors? When the answer is "I don't know"? Include examples of graceful handling.
- ☐ **Is the system prompt identical across all examples?** Varying the system prompt confuses the model. Pick one and use it everywhere.

Minimum data volumes (rules of thumb):

Examples	What You Can Expect
50–100	Basic style/format adoption with LoRA
200–500	Noticeable tone and structure consistency
500–2K	Reliable format + style change
2K–10K	Domain terminology adoption
10K–100K	Deep behavioral changes
100K+	Full domain specialization

Data Preparation Pipeline

A complete, runnable pipeline that takes raw data and produces training-ready JSONL:

```
"""
Fine-Tuning Data Preparation Pipeline
--------------------------------------
Takes raw data (CSV or JSON), converts to chat JSONL format,
validates, splits into train/validation, and reports statistics.

Usage:
    python prepare_data.py --input raw_data.csv --output ./output
"""

import json
import csv
import random
import argparse
from pathlib import Path
from collections import Counter

# ── Configuration ──────────────────────────────────────────

SYSTEM_PROMPT = (
    "You are a helpful assistant specialized in customer support. "
    "Respond concisely, professionally, and always suggest a next step."
)

# Approximate tokens ≈ words × 1.3 (rough estimate for English)
MAX_TOKENS_PER_EXAMPLE = 4096
MIN_ASSISTANT_TOKENS = 10
```

```python
TRAIN_SPLIT = 0.9


# ── Step 1: Load Raw Data ──────────────────────────────────────

def load_raw_data(filepath: str) -> list[dict]:
    """Load raw data from CSV or JSON."""
    path = Path(filepath)
    if path.suffix == ".csv":
        with open(path, "r", encoding="utf-8") as f:
            return list(csv.DictReader(f))
    elif path.suffix == ".json":
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    else:
        raise ValueError(f"Unsupported format: {path.suffix}. Use .csv or .json")


# ── Step 2: Convert to Chat Format ─────────────────────────────

def to_chat_format(row: dict) -> dict:
    """Convert a raw data row to OpenAI chat fine-tuning format.
    Expects 'input' and 'output' fields in the raw data.
    """
    return {
        "messages": [
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": row["input"].strip()},
            {"role": "assistant", "content": row["output"].strip()},
        ]
    }


# ── Step 3: Validate Each Example ──────────────────────────────

def estimate_tokens(text: str) -> int:
    """Rough token estimate: ~1.3 tokens per word for English."""
    return int(len(text.split()) * 1.3)


def validate_example(example: dict, idx: int) -> list[str]:
    """Validate a single training example. Returns list of errors."""
    errors = []
    messages = example.get("messages", [])

    # Check required roles
    roles = [m["role"] for m in messages]
    if "user" not in roles:
        errors.append(f"Example {idx}: missing 'user' message")
    if "assistant" not in roles:
        errors.append(f"Example {idx}: missing 'assistant' message")

    # Check for empty content
    for msg in messages:
        if not msg.get("content", "").strip():
            errors.append(f"Example {idx}: empty content for role '{msg['role']}'")
```

```
    # Check token count
    total_text = " ".join(m["content"] for m in messages)
    token_count = estimate_tokens(total_text)
    if token_count > MAX_TOKENS_PER_EXAMPLE:
        errors.append(f"Example {idx}: ~{token_count} tokens exceeds max {MAX_TOKENS

    # Check assistant response length
    for msg in messages:
        if msg["role"] == "assistant":
            asst_tokens = estimate_tokens(msg["content"])
            if asst_tokens < MIN_ASSISTANT_TOKENS:
                errors.append(f"Example {idx}: assistant response too short (~{asst_

    return errors


# ── Step 4: Split Train / Validation ──────────────────────────

def split_data(examples: list[dict], train_ratio: float = TRAIN_SPLIT):
    """Shuffle and split into train/validation sets."""
    shuffled = examples.copy()
    random.seed(42)
    random.shuffle(shuffled)
    split_idx = int(len(shuffled) * train_ratio)
    return shuffled[:split_idx], shuffled[split_idx:]


# ── Step 5: Report Statistics ─────────────────────────────────

def report_stats(examples: list[dict], label: str):
    """Print dataset statistics."""
    token_counts = []
    for ex in examples:
        total = " ".join(m["content"] for m in ex["messages"])
        token_counts.append(estimate_tokens(total))
    avg_tokens = sum(token_counts) / len(token_counts) if token_counts else 0
    # Cost estimate for OpenAI gpt-4o-mini fine-tuning (~$0.003 / 1K tokens)
    est_cost = sum(token_counts) * 0.000003 * 3  # 3 epochs
    print(f"\n{'='*50}")
    print(f"  {label}")
    print(f"{'='*50}")
    print(f"  Examples:     {len(examples)}")
    print(f"  Avg tokens:   {avg_tokens:.0f}")
    print(f"  Min tokens:   {min(token_counts)}")
    print(f"  Max tokens:   {max(token_counts)}")
    print(f"  Est. cost:    ${est_cost:.2f} (gpt-4o-mini, 3 epochs)")


def save_jsonl(examples: list[dict], filepath: str):
    """Write examples to a JSONL file."""
    with open(filepath, "w", encoding="utf-8") as f:
        for ex in examples:
            f.write(json.dumps(ex, ensure_ascii=False) + "\n")
    print(f"  Saved: {filepath} ({len(examples)} examples)")
```

```python
# ── Main ──────────────────────────────────────────────────────

def main():
    parser = argparse.ArgumentParser(description="Prepare fine-tuning data")
    parser.add_argument("--input", required=True, help="Path to raw CSV or JSON")
    parser.add_argument("--output", default="./output", help="Output directory")
    args = parser.parse_args()

    # Load
    print("Loading raw data...")
    raw = load_raw_data(args.input)
    print(f"  Loaded {len(raw)} raw examples")

    # Convert
    print("Converting to chat format...")
    examples = [to_chat_format(row) for row in raw]

    # Validate
    print("Validating...")
    all_errors = []
    valid_examples = []
    for i, ex in enumerate(examples):
        errs = validate_example(ex, i)
        if errs:
            all_errors.extend(errs)
        else:
            valid_examples.append(ex)
    print(f"  Valid: {len(valid_examples)} | Invalid: {len(examples) - len(valid_exa
    for err in all_errors[:10]:
        print(f"  ⚠ {err}")
    if len(all_errors) > 10:
        print(f"  ... and {len(all_errors) - 10} more errors")

    # Split
    train, val = split_data(valid_examples)

    # Save
    out = Path(args.output)
    out.mkdir(parents=True, exist_ok=True)
    save_jsonl(train, out / "train.jsonl")
    save_jsonl(val, out / "val.jsonl")

    # Stats
    report_stats(train, "Training Set")
    report_stats(val, "Validation Set")


if __name__ == "__main__":
    main()
```

Common Data Mistakes

```
Mistake #1: INCONSISTENT FORMATTING
┌──────────────────────────────────────────────────┐
│ Example 1 assistant: "Category: BILLING"         │
│ Example 2 assistant: "The category is billing."  │
│ Example 3 assistant: "billing"                   │
│                                                  │
│ The model learns ALL of these formats and picks  │
│ randomly at inference time. Pick ONE format.     │
└──────────────────────────────────────────────────┘

Mistake #2: TOO MANY SIMILAR EXAMPLES
  If 80% of your data is about password resets, the model
  becomes a password-reset specialist and forgets everything
  else. Balance your categories.

Mistake #3: MISSING EDGE CASES
  You trained on 1000 "normal" inputs. User sends garbage.
  Model hallucinates confidently. Include examples of how
  to handle unexpected/invalid/ambiguous inputs.

Mistake #4: VARYING SYSTEM PROMPTS
  Example 1: "You are a helpful assistant."
  Example 2: "You are an expert customer service agent."
  Example 3: "Answer questions about our product."
  → Pick ONE system prompt and use it for ALL examples.

Mistake #5: MIXED QUALITY
  Some responses are excellent, some are mediocre.
  The model learns the AVERAGE quality. Every single
  response should be gold-standard.
```

Fine-Tuning Methods (Practical)

There are three practical paths to fine-tuning today. Think of them as different levels of control over your kitchen:

```
OpenAI API       = Ordering from a restaurant. You describe what you want,
                   the chef makes it. Easy but limited customization.

LoRA (Hugging Face) = Cooking in your own kitchen with a recipe.
                      Full control, need your own equipment.

QLoRA            = Same kitchen, but with a smaller stove.
                   Budget-friendly, nearly the same results.
```

OpenAI Fine-Tuning (API-based)

The fastest path from zero to fine-tuned model. No GPU needed — OpenAI handles all the infrastructure. Best for quick experiments and production use with OpenAI models.

```
from openai import OpenAI

client = OpenAI()  # uses OPENAI_API_KEY env var

# ── Step 1: Upload your training file ──────────────────────

training_file = client.files.create(
    file=open("output/train.jsonl", "rb"),
    purpose="fine-tune"
)
print(f"Uploaded: {training_file.id}")

# ── Step 2: (Optional) Upload validation file ──────────────

val_file = client.files.create(
    file=open("output/val.jsonl", "rb"),
    purpose="fine-tune"
)

# ── Step 3: Create fine-tuning job ─────────────────────────

job = client.fine_tuning.jobs.create(
    training_file=training_file.id,
    validation_file=val_file.id,
    model="gpt-4o-mini-2024-07-18",   # base model
    hyperparameters={
        "n_epochs": 3,                 # start with 3, adjust based on val loss
        "learning_rate_multiplier": 1.8,
        "batch_size": "auto",
    },
    suffix="my-custom-task"            # appears in model name
)
print(f"Job created: {job.id}")

# ── Step 4: Monitor training progress ──────────────────────

import time

while True:
    status = client.fine_tuning.jobs.retrieve(job.id)
    print(f"Status: {status.status}")
    if status.status in ("succeeded", "failed", "cancelled"):
        break
    time.sleep(60)

# Check training events for loss curves
events = client.fine_tuning.jobs.list_events(
    fine_tuning_job_id=job.id,
    limit=20
)
for event in events.data:
```

```
    print(f"  {event.message}")

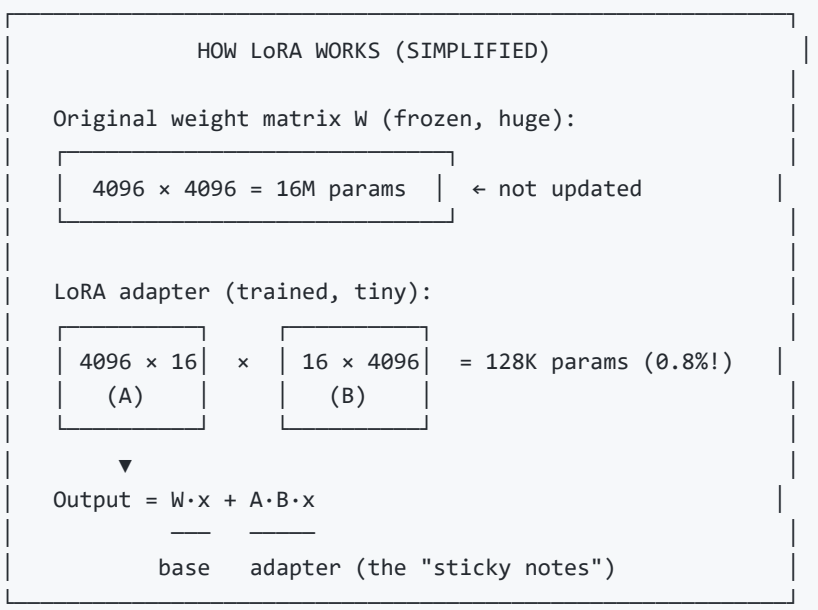
# ── Step 5: Use your fine-tuned model ──────────────────────

fine_tuned_model = status.fine_tuned_model  # e.g. "ft:gpt-4o-mini:my-org:my-custom-

response = client.chat.completions.create(
    model=fine_tuned_model,
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "How do I reset my password?"},
    ]
)
print(response.choices[0].message.content)
```

LoRA with Hugging Face (Open-Source)

Analogy: LoRA (Low-Rank Adaptation) is like adding sticky notes to a textbook instead of rewriting it. The original book (base model weights) stays frozen. You only train the sticky notes (small adapter matrices), which are ~1-5% of the total parameters.



Full training workflow with `transformers` + `peft` + `trl`:

```
import torch
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
)
```

```python
from peft import LoraConfig, get_peft_model, TaskType
from trl import SFTTrainer

# ── Step 1: Load base model and tokenizer ──────────────────────

model_name = "meta-llama/Llama-3.1-8B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.bfloat16,
    device_map="auto",
)

# ── Step 2: Configure LoRA ─────────────────────────────────────

lora_config = LoraConfig(
    r=16,                              # rank — capacity of the adapter
    lora_alpha=32,                     # scaling factor (typically 2× r)
    target_modules=[                   # which layers to adapt
        "q_proj", "k_proj", "v_proj",  # attention projections
        "o_proj",                      # output projection
        "gate_proj", "up_proj",        # MLP layers
        "down_proj",
    ],
    lora_dropout=0.05,                 # regularization
    bias="none",                       # don't train biases
    task_type=TaskType.CAUSAL_LM,
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# → trainable params: 13.6M || all params: 8B || trainable%: 0.17%

# ── Step 3: Load dataset ───────────────────────────────────────

dataset = load_dataset("json", data_files={
    "train": "output/train.jsonl",
    "validation": "output/val.jsonl",
})

# ── Step 4: Training arguments ─────────────────────────────────

training_args = TrainingArguments(
    output_dir="./lora-checkpoints",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,     # effective batch size = 16
    learning_rate=2e-4,
    warmup_ratio=0.05,
    weight_decay=0.01,
    bf16=True,
    logging_steps=10,
    eval_strategy="steps",
```

```
    eval_steps=50,
    save_strategy="steps",
    save_steps=100,
    load_best_model_at_end=True,
    report_to="none",
)

# ── Step 5: Train ──────────────────────────────────────

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],
    processing_class=tokenizer,
    peft_config=lora_config,
    max_seq_length=2048,
)

trainer.train()

# ── Step 6: Save adapter weights ──────────────────────

model.save_pretrained("./my-lora-adapter")
tokenizer.save_pretrained("./my-lora-adapter")

# To merge LoRA into base model for deployment:
# merged = model.merge_and_unload()
# merged.save_pretrained("./my-merged-model")
```

QLoRA (Budget-Friendly)

What's different: The base model is loaded in 4-bit precision, slashing memory by ~4×. LoRA adapters are still trained in BF16. You get nearly the same quality as full LoRA at a fraction of the GPU cost.

```
Memory comparison for LLaMA 3.1 8B:

  Full precision (FP32):  ~32 GB  ← needs 2× A100
  Half precision (BF16):  ~16 GB  ← needs 1× A100
  QLoRA (4-bit + LoRA):   ~6 GB   ← fits on RTX 4090 or even RTX 3090!
```

```
  import torch
  from transformers import AutoModelForCausalLM, BitsAndBytesConfig

  # ── 4-bit quantization config ──────────────────────────

  bnb_config = BitsAndBytesConfig(
      load_in_4bit=True,
      bnb_4bit_quant_type="nf4",          # NormalFloat4 — best for LLMs
```

```
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,     # quantize the quantization constants
)

# Load the model in 4-bit
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    quantization_config=bnb_config,
    device_map="auto",
)

# Now apply LoRA on top (same config as before)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

model = prepare_model_for_kbit_training(model)  # freeze + prep for training

lora_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
)
model = get_peft_model(model, lora_config)

# Training proceeds identically to the LoRA example above.
# The only difference is the base model is in 4-bit.
```

Method Comparison

	OpenAI API	LoRA (HF)	QLoRA (HF)
Effort	Low	Medium	Medium
Cost	Pay per token	GPU rental	GPU rental
GPU needed	None	1–2× A100	1× RTX 4090
Model access	Closed	Full weights	Full weights
Customization	Limited	Full control	Full control
Best for	Quick tests	Production	Budget

When to choose which:

- **OpenAI API** → You want something working in an hour, you're already using OpenAI, and you don't need model weights. Great for prototyping and moderate-scale production.
- **LoRA** → You need full control, want to self-host, or are using an open model (LLaMA 4, Mistral). The production standard for open-source fine-tuning.

- **QLoRA** → Same as LoRA but your GPU budget is limited. Quality is within ~1-2% of full LoRA for most tasks. Start here if you have a single consumer GPU.

Hyperparameter Guide

Hyperparameters control *how* the model learns. Bad hyperparameters can ruin even the best dataset. But don't overthink it — the defaults below work for 90% of cases.

Analogy: Hyperparameters are like the settings on a stovetop. Too high (high learning rate) = burnt food (catastrophic forgetting). Too low (low learning rate) = raw food (no learning). The trick is finding the right temperature and cooking time for your dish.

Key Hyperparameters

```
┌────────────────────┬────────────────────┬────────────────────┬──────────────────────────────┐
│ Parameter          │ LoRA Range         │ Full FT Range      │ If Wrong...                  │
├────────────────────┼────────────────────┼────────────────────┼──────────────────────────────┤
│ Learning rate      │ 1e-4 to 3e-4       │ 1e-5 to 5e-5       │ Too high → forgets           │
│                    │                    │                    │ Too low → doesn't learn      │
│ Epochs             │ 3–10 (small data)  │ 1–3 (large data)   │ Too many → overfits          │
│ Batch size         │ 4–16 (effective)   │ 16–64 (effective)  │ Too small → noisy            │
│ LoRA rank (r)      │ 8–64               │ N/A                │ Too low → underfits          │
│ Warmup ratio       │ 0.03–0.1           │ 0.03–0.1           │ Too low → unstable           │
│ Weight decay       │ 0.01–0.1           │ 0.01–0.1           │ Too high → underfits         │
└────────────────────┴────────────────────┴────────────────────┴──────────────────────────────┘
```

LoRA rank (r) — choosing the right value:

```
Think of rank as the adapter's "vocabulary size" for learning new behavior:

  r = 4–8     → "I just need to change the formatting style."
                Minimal adaptation. Fewest trainable parameters.

  r = 16–32   → "I need the model to adopt domain terminology and
                follow specific workflows." This is the sweet spot
                for most fine-tuning tasks.

  r = 64–128  → "I need significant behavioral changes or deep
                domain knowledge." Rare. Consider if LoRA is enough
                or if full fine-tuning is needed.
```

Gradient accumulation — simulating large batches on small GPUs:

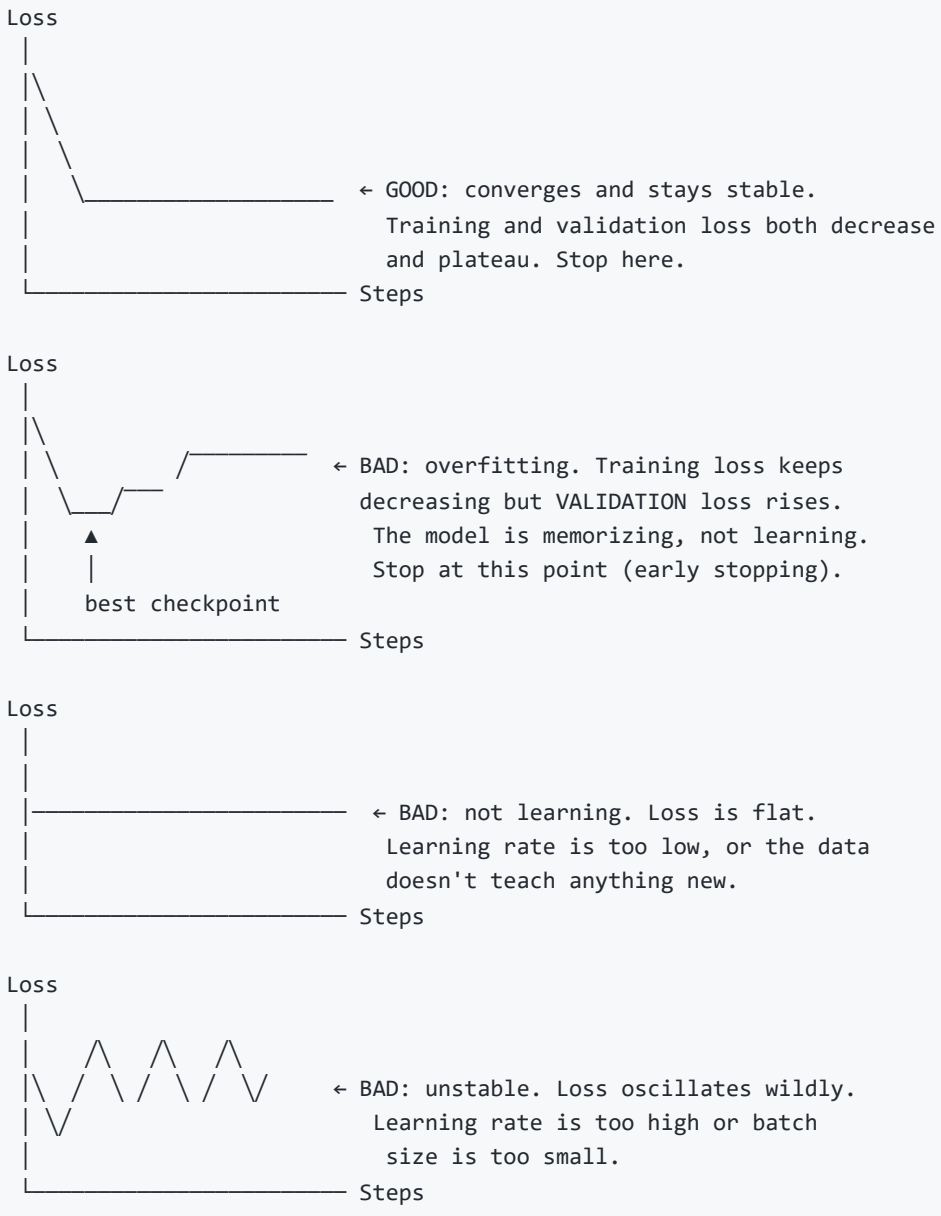
```
Can't fit batch_size=16 in GPU memory?
Use gradient accumulation:
```

```
  per_device_batch_size = 4
  gradient_accumulation_steps = 4
  effective_batch_size = 4 × 4 = 16  ✓

The model processes 4 examples at a time but only updates
weights every 4 mini-batches. Mathematically equivalent to
batch_size=16 (with slightly more noise, which is often fine).
```

Reading the Loss Curve

The training loss curve tells you almost everything about how training is going. Learn to read it:

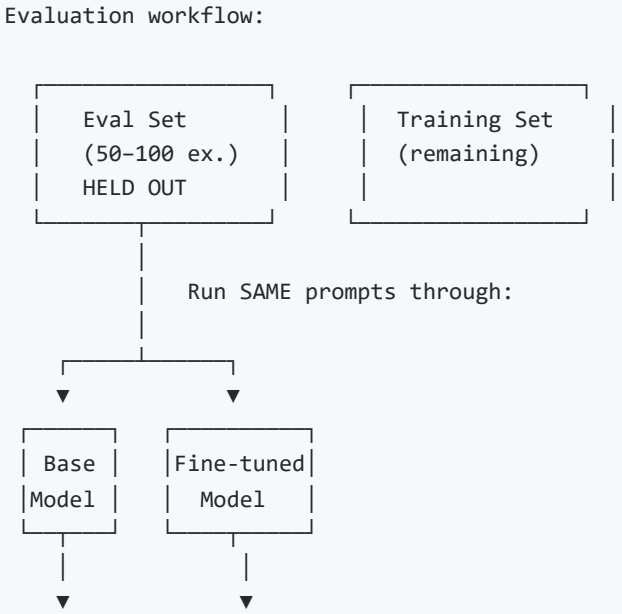


Evaluation & Testing

Training a model without evaluating it is like baking a cake without tasting it. You might get lucky, or you might serve garbage with confidence.

Before vs After Comparison

The golden rule: **build your eval set BEFORE you start training**. Never, ever train on eval data.



Automated Evaluation

Different tasks need different metrics. Pick the right one:

- **Exact match / regex** — For structured outputs (JSON, classifications, code). Did the model produce the exact expected format? Parse the output and check programmatically.
- **LLM-as-judge** — Use a strong model (GPT-5, Claude) to rate outputs 1–5 on criteria like helpfulness, accuracy, and tone. Surprisingly reliable and scales well.
- **Domain metrics** — F1 score for classification, BLEU/ROUGE for text generation, pass@k for code generation. Use the standard metric for your task type.

```
"""
Evaluation Script — Compare base model vs fine-tuned model.
Uses LLM-as-judge for quality scoring.
"""
```

```python
import json
from openai import OpenAI

client = OpenAI()

JUDGE_SYSTEM_PROMPT = """You are an expert evaluator. You will be given:
1. A user query
2. A gold-standard reference answer
3. A model-generated answer

Rate the model answer from 1 to 5:
  5 = Perfect — matches or exceeds the reference
  4 = Good — minor differences, still correct and useful
  3 = Acceptable — mostly correct but missing key details
  2 = Poor — significant errors or missing information
  1 = Bad — wrong, unhelpful, or harmful

Respond with ONLY a JSON object: {"score": <int>, "reason": "<brief explanation>"}"""


def judge_response(query: str, reference: str, model_answer: str,
                   judge_model: str = "gpt-5.1") -> dict:
    """Use an LLM to score a model's response against a reference."""
    response = client.chat.completions.create(
        model=judge_model,
        messages=[
            {"role": "system", "content": JUDGE_SYSTEM_PROMPT},
            {"role": "user", "content": (
                f"Query: {query}\n\n"
                f"Reference answer: {reference}\n\n"
                f"Model answer: {model_answer}"
            )},
        ],
        temperature=0.0,
    )
    return json.loads(response.choices[0].message.content)


def evaluate_model(model_id: str, eval_path: str,
                   judge_model: str = "gpt-5.1") -> dict:
    """Run evaluation on a model using held-out eval set."""
    with open(eval_path, "r") as f:
        eval_set = [json.loads(line) for line in f]

    results = []
    for example in eval_set:
        msgs = example["messages"]
        user_msg = next(m["content"] for m in msgs if m["role"] == "user")
        reference = next(m["content"] for m in msgs if m["role"] == "assistant")
        system_msg = next(
            (m["content"] for m in msgs if m["role"] == "system"), None
        )

        # Get model response
        prompt = [{"role": "user", "content": user_msg}]
        if system_msg:
```

```
            prompt.insert(0, {"role": "system", "content": system_msg})

        completion = client.chat.completions.create(
            model=model_id, messages=prompt, temperature=0.0,
        )
        model_answer = completion.choices[0].message.content

        # Judge it
        verdict = judge_response(user_msg, reference, model_answer, judge_model)
        verdict["query"] = user_msg[:100]
        verdict["model_answer"] = model_answer[:200]
        results.append(verdict)

    # Summary
    scores = [r["score"] for r in results]
    summary = {
        "model": model_id,
        "num_examples": len(results),
        "avg_score": sum(scores) / len(scores),
        "score_distribution": {i: scores.count(i) for i in range(1, 6)},
        "results": results,
    }
    print(f"\n{'='*50}")
    print(f"  Model: {model_id}")
    print(f"  Avg Score: {summary['avg_score']:.2f} / 5.0")
    print(f"  Distribution: {summary['score_distribution']}")
    print(f"{'='*50}")
    return summary

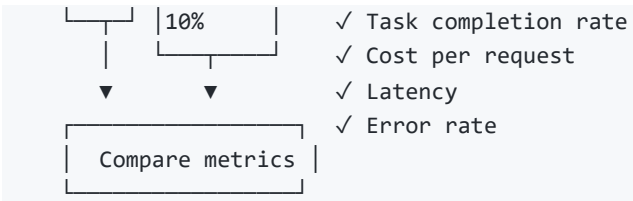

# ── Usage ──────────────────────────────────────────────────
# base_results = evaluate_model("gpt-4o-mini", "output/val.jsonl")
# ft_results   = evaluate_model("ft:gpt-4o-mini:org:task:id", "output/val.jsonl")
# Compare base_results["avg_score"] vs ft_results["avg_score"]
```

A/B Testing in Production

Lab results don't always match production. Once your eval looks good, A/B test with real traffic:

```
Production A/B test setup:

  User request
       │
       ▼
  ┌──────────┐
  │  Router  │
  │ (90/10)  │
  └──┬────┬──┘
     │    │
     ▼    ▼
  ┌────┐ ┌──────┐
  │Base│ │Fine-  │    Measure:
  │90% │ │tuned  │    ✓ User satisfaction
```



- Start at 10% traffic to the fine-tuned model
- Run for at least 1–2 weeks to get statistical significance
- If metrics improve → gradually ramp to 50%, then 100%
- If metrics are flat → fine-tuning didn't help, revert and save the cost
- If metrics degrade → stop immediately, investigate regressions

Red-Teaming

Fine-tuning can introduce new failure modes that the base model didn't have. Test for them explicitly:

- **Catastrophic forgetting** — Can the model still do general tasks? Ask it to summarize an article, write an email, do basic math. If it can't, the learning rate was too high or you trained too many epochs.
- **Adversarial inputs** — Send malicious, confusing, or off-topic inputs. Does the model handle them gracefully or does it hallucinate? Include a few adversarial examples in your training data.
- **Out-of-distribution inputs** — Send inputs that are valid but very different from training data. A model fine-tuned on English should still respond sensibly to a mixed-language input.
- **Regression on known-good inputs** — Keep a set of inputs the base model already handled well. Verify fine-tuning didn't break them.

Best Practices & Pitfalls

Best Practices

- **Start with prompting** — Fine-tune only when prompting provably fails. Document the failure cases so you can evaluate whether fine-tuning actually fixed them.
- **Data quality > quantity** — 500 perfect examples beat 50,000 noisy ones. Manually review at least 10% of your training data. If you wouldn't ship that response to a customer, fix it.
- **Keep a held-out eval set** — Never train on your test data. Split the data BEFORE any processing or cleaning. Use the eval set to compare base model vs fine-tuned model.

- **Version everything** — Data files, hyperparameters, model checkpoints, eval results. You will need to reproduce or roll back. Use git for data, log hyperparams to a file, save checkpoints.
- **Start with LoRA** — Only do full fine-tuning if LoRA provably isn't enough. LoRA is 10–100× cheaper and usually gets you 90%+ of the way there.
- **Use the smallest model that works** — A fine-tuned 8B model often beats a prompted 70B model for specific tasks, at 1/10th the inference cost. Don't default to the biggest model.
- **Monitor for regression** — Fine-tuning can break capabilities the base model had. Always test general capabilities alongside your specific task.
- **Document what changed** — Future you (or your teammate) needs to know: why you fine-tuned, what data you used, what hyperparams worked, and what the eval results showed.

Common Pitfalls

- **Catastrophic forgetting**: The model loses general abilities after fine-tuning. *Prevention*: Use a low learning rate (1e-4 to 2e-4 for LoRA), train for few epochs (2–3), and mix in 5–10% of general-purpose data in your training set.

- **Overfitting**: The model memorizes training examples instead of learning patterns. *Symptoms*: Training loss goes to zero, validation loss rises. *Prevention*: More diverse data, early stopping, regularization (dropout 0.05, weight decay 0.01).

- **Data leakage**: Eval data appears in the training set, giving false confidence in your results. *Prevention*: Split your data into train/eval BEFORE any cleaning or augmentation. Use deterministic splits (seed=42).

- **Inconsistent data**: Mixed formats or styles across examples confuse the model. The model learns the average, which satisfies nobody. *Prevention*: Run the data validation pipeline, enforce a schema, review samples.

- **Training too long**: More epochs ≠ better. After 2–3 epochs on a well-curated dataset, additional training almost always hurts. *Prevention*: Watch validation loss. Stop when it plateaus or starts rising.

- **Wrong base model**: Don't fine-tune a 70B model if an 8B model would do. Cost scales linearly with parameter count, and larger models are harder to deploy. *Rule of thumb*: Start with the smallest model, scale up only if eval metrics demand it.

Cost Estimation

```
┌──────────────────────┬──────────────┬───────────────┬──────────────┐
│ Method               │ ~1K examples │ ~10K examples │ ~100K examples│
├──────────────────────┼──────────────┼───────────────┼──────────────┤
│ OpenAI (gpt-4o-mini)│ ~$2–5        │ ~$20–50       │ ~$200–500    │
│ LoRA (A100 rental)   │ ~$5–10       │ ~$10–30       │ ~$50–200     │
```

```
| QLoRA (RTX 4090)    | ~$2–5          | ~$5–15          | ~$20–100     |
| Full FT (8× A100)   | ~$50–100       | ~$100–500       | ~$500–5000   |
└─────────────────────┴────────────────┴─────────────────┴──────────────┘

Note: GPU rental costs assume cloud pricing (~$2/hr for A100, ~$0.50/hr
for RTX 4090 via Lambda, RunPod, or Vast.ai). Costs decrease over time
as hardware prices drop and competition increases.
```

Complete Workflow Checklist

Use this end-to-end checklist every time you fine-tune. Don't skip steps — they exist because someone learned the hard way.

```
┌──────────────────────────────────────────────────────────────┐
│              FINE-TUNING WORKFLOW CHECKLIST                  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  □ 1. CONFIRM fine-tuning is the right approach              │
│       → Have you tried prompting? RAG? A bigger model?       │
│       → Can you articulate WHY fine-tuning is needed?        │
│                                                              │
│  □ 2. COLLECT and curate training data                       │
│       → Quality > quantity. Review every example.            │
│       → Minimum: 100 for format, 500+ for behavior.          │
│                                                              │
│  □ 3. FORMAT data as JSONL, validate, split train/eval       │
│       → Use the data preparation pipeline above.             │
│       → Consistent system prompt across ALL examples.        │
│                                                              │
│  □ 4. CHOOSE method: OpenAI API vs LoRA vs QLoRA             │
│       → Start with OpenAI API for speed, LoRA for control.   │
│                                                              │
│  □ 5. SET hyperparameters (start conservative)               │
│       → LoRA: lr=2e-4, epochs=3, r=16, warmup=0.05           │
│       → Full FT: lr=2e-5, epochs=1-2, warmup=0.05            │
│                                                              │
│  □ 6. TRAIN and monitor loss curves                          │
│       → Watch for overfitting (val loss rising).             │
│       → Save checkpoints every 100 steps.                    │
│                                                              │
│  □ 7. EVALUATE on held-out set                               │
│       → Compare to base model (side-by-side).                │
│       → Use LLM-as-judge + domain-specific metrics.          │
│                                                              │
│  □ 8. RED-TEAM for regressions and edge cases                │
│       → Test general capabilities (catastrophic forgetting). │
│       → Test adversarial and out-of-distribution inputs.     │
│                                                              │
│  □ 9. A/B TEST in production (10% traffic)                   │
│       → Measure real user outcomes, not just lab metrics.    │
│       → Run for 1–2 weeks minimum.                           │
```

```
│                                                                  │
│  □ 10. DOCUMENT everything                                       │
│       → Data source, size, and version.                          │
│       → Hyperparameters and training logs.                       │
│       → Eval results (base vs fine-tuned).                       │
│       → Decisions and reasoning for future reference.            │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

← Back to AI & Agents Syllabus

Related → Day 1 Appendix: Advanced Techniques (LoRA, Quantization)